Universidad Galileo
Postgrado en Desarrollo de Software - Ciberseguridad

# Laboratorio: Análisis SCA local y en pipeline con Dependency-Check

Aplicación analizada: JavaVulnerableLab

Fecha: 30 de octubre de 2026

**Enlaces**

| | |
|---|---|
| Repositorio | https://github.com/pagreda888/javavulnerablelab-sca |
| Workflow | sca-dependency-check.yml |
| Pull Request (remediación) | https://github.com/pagreda888/javavulnerablelab-sca/pull/1 |
| Ejecuciones del pipeline | https://github.com/pagreda888/javavulnerablelab-sca/actions |
| Code scanning | https://github.com/pagreda888/javavulnerablelab-sca/security/code-scanning |

## Parte A - Repositorio propio

Cloné JavaVulnerableLab, eliminé la carpeta .git y creé un repositorio nuevo y público llamado javavulnerablelab-sca en mi cuenta de GitHub (pagreda888). En el .gitignore ya venía target/ y le agregué las carpetas de reportes locales.

Al hacer el primer push, GitHub Push Protection bloqueó la subida porque detectó claves con formato de Stripe en dos archivos (backup/config.properties.bak e integrations.jsp). Revisé los archivos y son secretos falsos que vienen a propósito en la aplicación vulnerable, así que los permití desde el enlace que da GitHub.

Repositorio: https://github.com/pagreda888/javavulnerablelab-sca

### Dependencias declaradas en el pom.xml:

| Dependencia | Versión | Observación |
|---|---|---|
| junit:junit | 3.8.1 | scope test |
| mysql:mysql-connector-java | 5.1.26 | |
| org.json:json | 20090211 | |
| javax.servlet:jstl | 1.2 | |
| org.hibernate:hibernate-core | 4.0.1.Final | trae varias transitivas (dom4j, antlr, javassist...) |
| javax.servlet:servlet-api | 2.3 | scope provided |
| org.apache.logging.log4j:log4j-api | 2.12.1 | |
| org.apache.logging.log4j:log4j-core | 2.12.1 | vulnerable a Log4Shell (CVE-2021-44228) |
| commons-collections:commons-collections | 3.1 | gadget de deserialización |
| commons-fileupload:commons-fileupload | 1.3.3 | |
| commons-io:commons-io | 2.6 | |

## Parte B - Dependency-Check en local

Lo ejecuté en macOS con Java 17 y Maven instalados con Homebrew. La clave del NVD la pasé como variable de entorno para no escribirla en ningún archivo:

```
export NVD_API_KEY="mi-clave"
mvn -B org.owasp:dependency-check-maven:13.0.0:check "-Dformat=ALL" \
    "-DnvdApiKey=$NVD_API_KEY" "-DossindexAnalyzerEnabled=false"
```

La primera ejecución tardó casi una hora (53 min) porque descargó la base completa del NVD, unos 417 mil registros. Las siguientes tardan alrededor de un minuto porque la base queda en el repositorio local de Maven (~/.m2/repository/org/owasp/).

Resultado: 19 dependencias analizadas, 9 vulnerables y 31 vulnerabilidades en total.

| Dependencia | Tipo | Severidad máx. | CVEs |
|---|---|---|---|
| log4j-core 2.12.1 | directa | CRITICAL | CVE-2021-44228 (Log4Shell), CVE-2021-45046, CVE-2021-45105, CVE-2021-44832, CVE-2020-9488, CVE-2025-68161, CVE-2026-34477, CVE-2026-34479, CVE-2026-34480 |

| Dependencia | Tipo | Severidad máx. | CVEs |
|---|---|---|---|
| commons-collections 3.1 | directa | CRITICAL | CVE-2015-6420 |
| dom4j 1.6.1 | transitiva | CRITICAL | CVE-2020-10683 |
| mysql-connector-java 5.1.26 | directa | HIGH | CVE-2023-22102, CVE-2019-2692, CVE-2020-2934, CVE-2020-2875, CVE-2020-2933 |
| commons-fileupload 1.3.3 | directa | HIGH | CVE-2023-24998, CVE-2025-48976 |
| hibernate-core 4.0.1.Final | directa | HIGH | CVE-2020-25638, CVE-2019-14900 |
| commons-io 2.6 | directa | MEDIUM | CVE-2021-29425, CVE-2024-47554 |
| log4j-api 2.12.1 | directa | MEDIUM | CVE-2020-9488, CVE-2026-34477, CVE-2026-34479 |
| jquery 1.6.4 (jquery.min.js) | archivo JS | MEDIUM | CVE-2012-6708, CVE-2015-9251, CVE-2019-11358, CVE-2020-11023, CVE-2020-7656 |

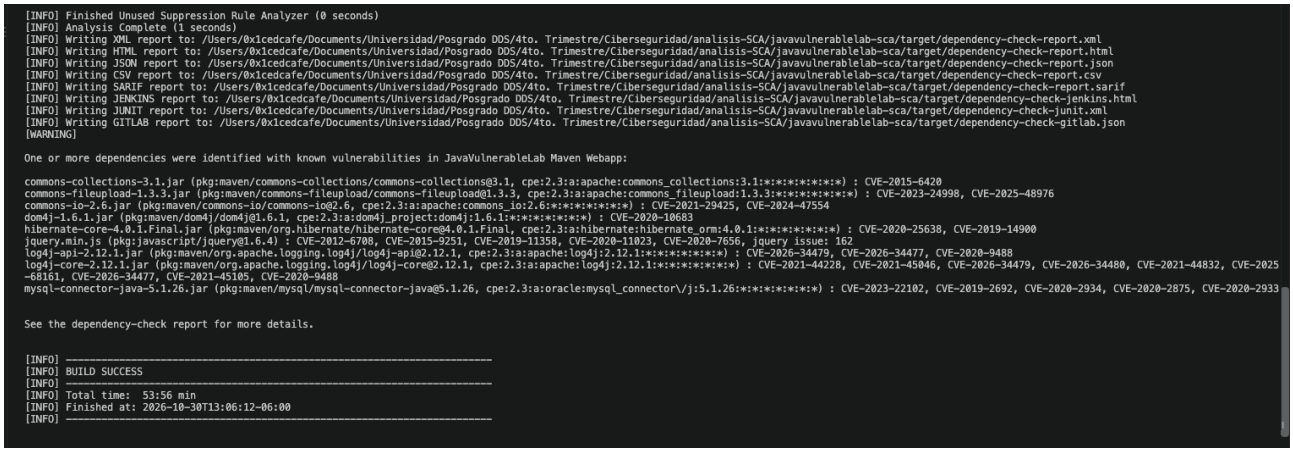



*Figura 1. Ejecución local de Dependency-Check en la terminal.*

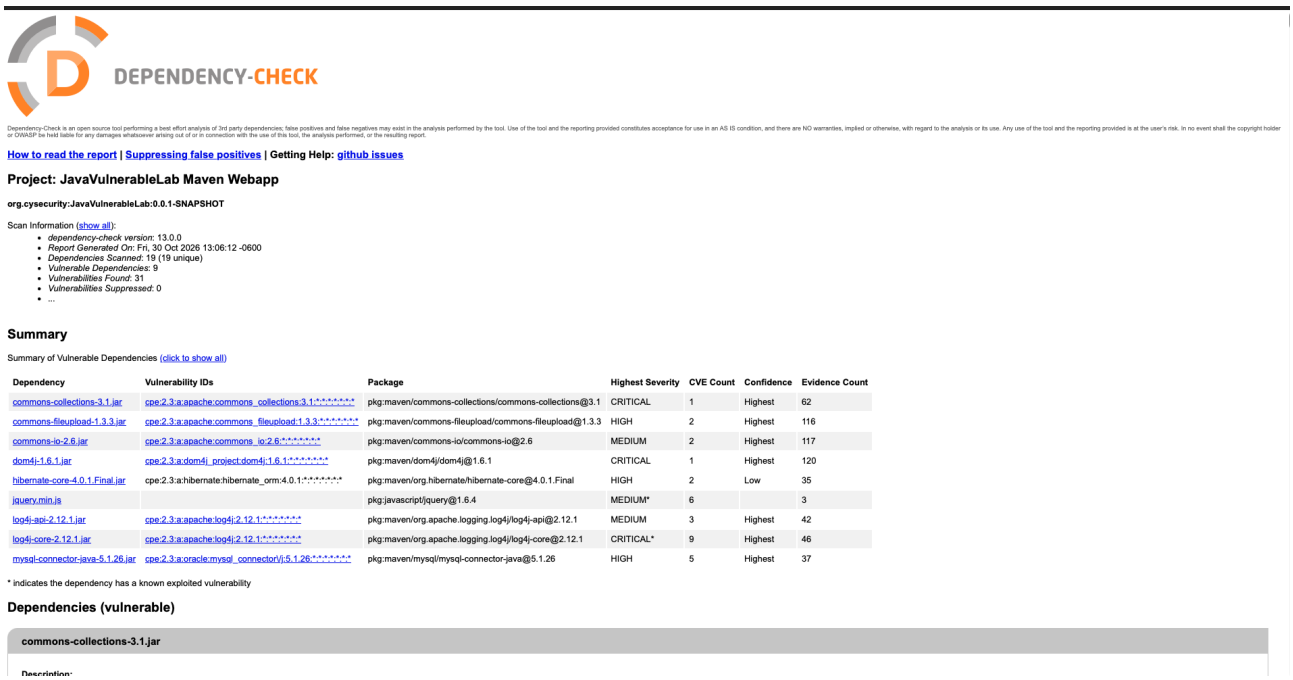



*Figura 2. Reporte HTML (target/dependency-check-report.html).*

## Parte C - SBOM y Dependency-Track

Generé el SBOM en formato CycloneDX JSON con el plugin de Maven:

```
mvn -B org.cyclonedx:cyclonedx-maven-plugin:2.9.1:makeAggregateBom
```

El archivo target/bom.json tiene **18 componentes**: 10 son dependencias directas y 8 son transitivas (antlr, dom4j, xml-apis, jboss-logging, javassist, hibernate-jpa-2.0-api, hibernate-commons-annotations y jboss-transaction-api). junit no aparece porque es de scope test.

Levanté Dependency-Track con Docker, creé el proyecto JavaVulnerableLab versión 1.0 y subí el bom.json. Al principio el proyecto salía con 0 vulnerabilidades. Tuve que activar Google OSV (ecosistema Maven) en Administration > Vulnerability Sources y volver a analizar. Después de eso aparecieron **29 vulnerabilidades**: 4 críticas, 12 altas, 11 medias y 2 bajas.

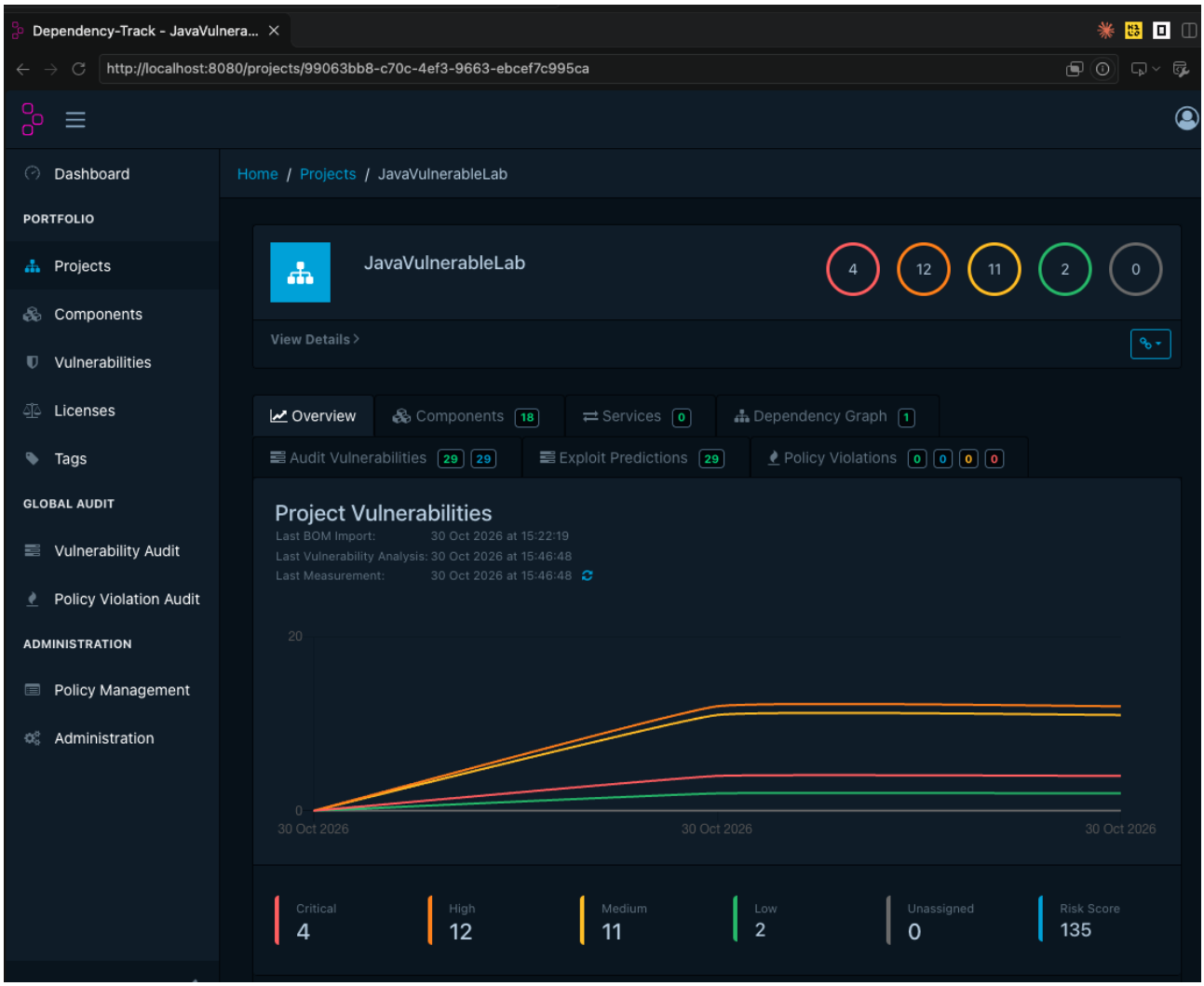



*Figura 3. Dependency-Track - Overview del proyecto.*

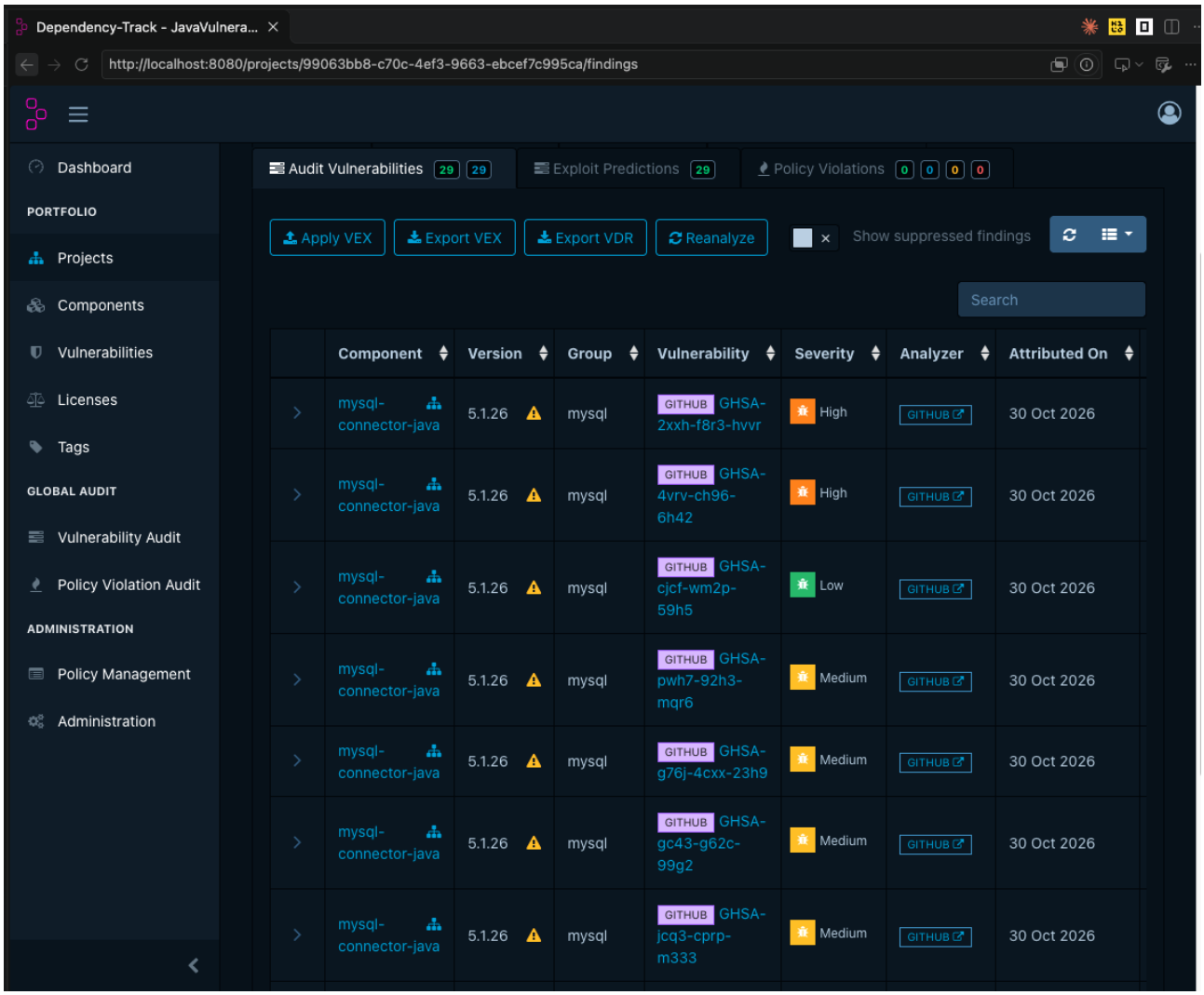



*Figura 4. Dependency-Track - Audit Vulnerabilities.*

## Dependencia transitiva vulnerable (Dependency Graph)

En el Dependency Graph se ve que **dom4j 1.6.1** no está en el pom.xml, sino que llega porque **hibernate-core 4.0.1.Final** la necesita. Esa versión de dom4j tiene la CVE-2020-10683 (XXE, crítica). O sea que el proyecto queda expuesto aunque nunca se declaró dom4j. Pasa lo mismo con commons-io, que además de estar declarada también la trae commons-fileupload.

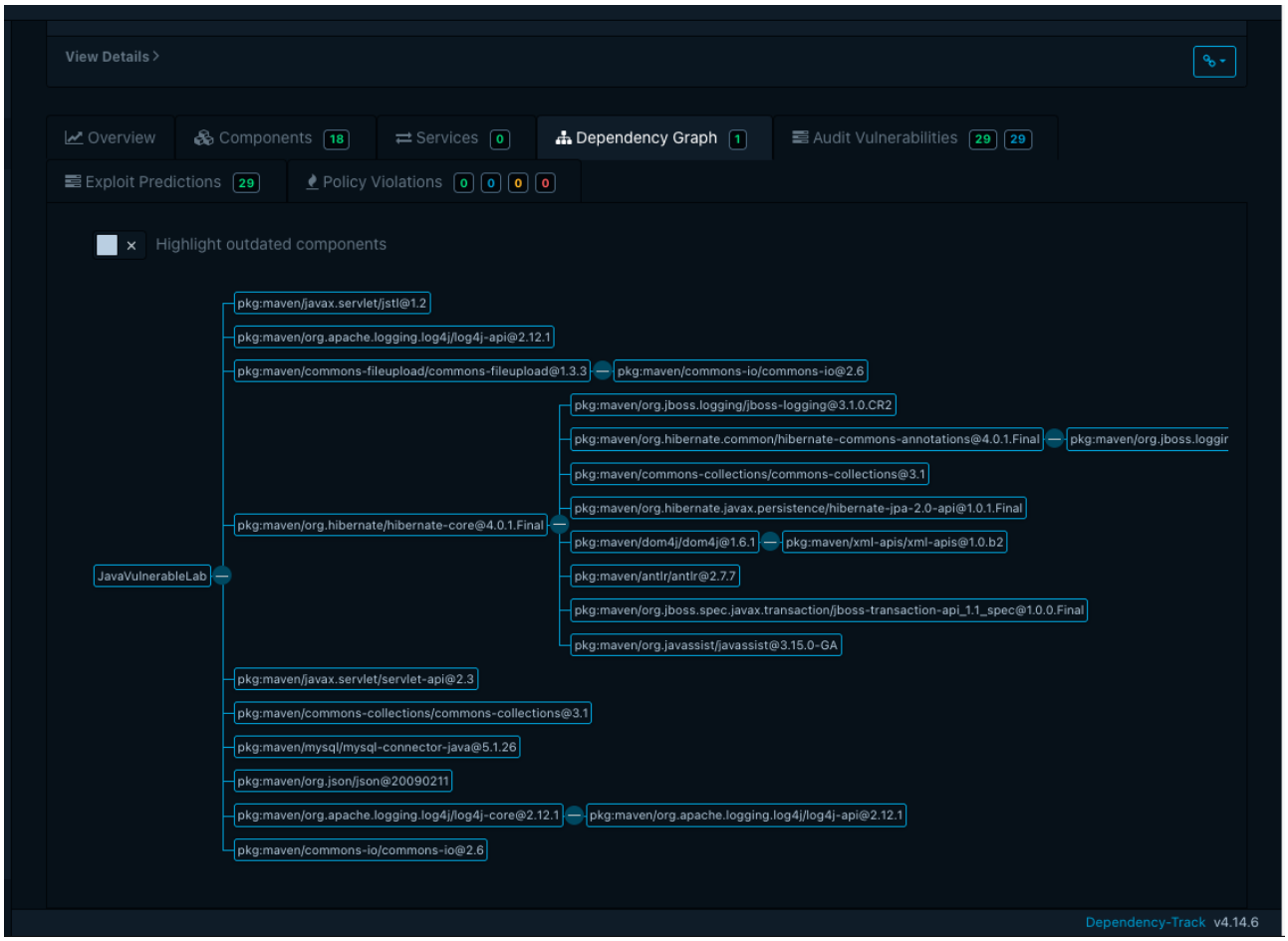



*Figura 5. Dependency Graph: dom4j 1.6.1 cuelga de hibernate-core 4.0.1.Final.*

## Parte D - DefectDojo y comparación

En DefectDojo creé el producto JavaVulnerableLab, un engagement y ahí importé target/dependency-check-report.xml con el tipo de escaneo Dependency Check Scan.

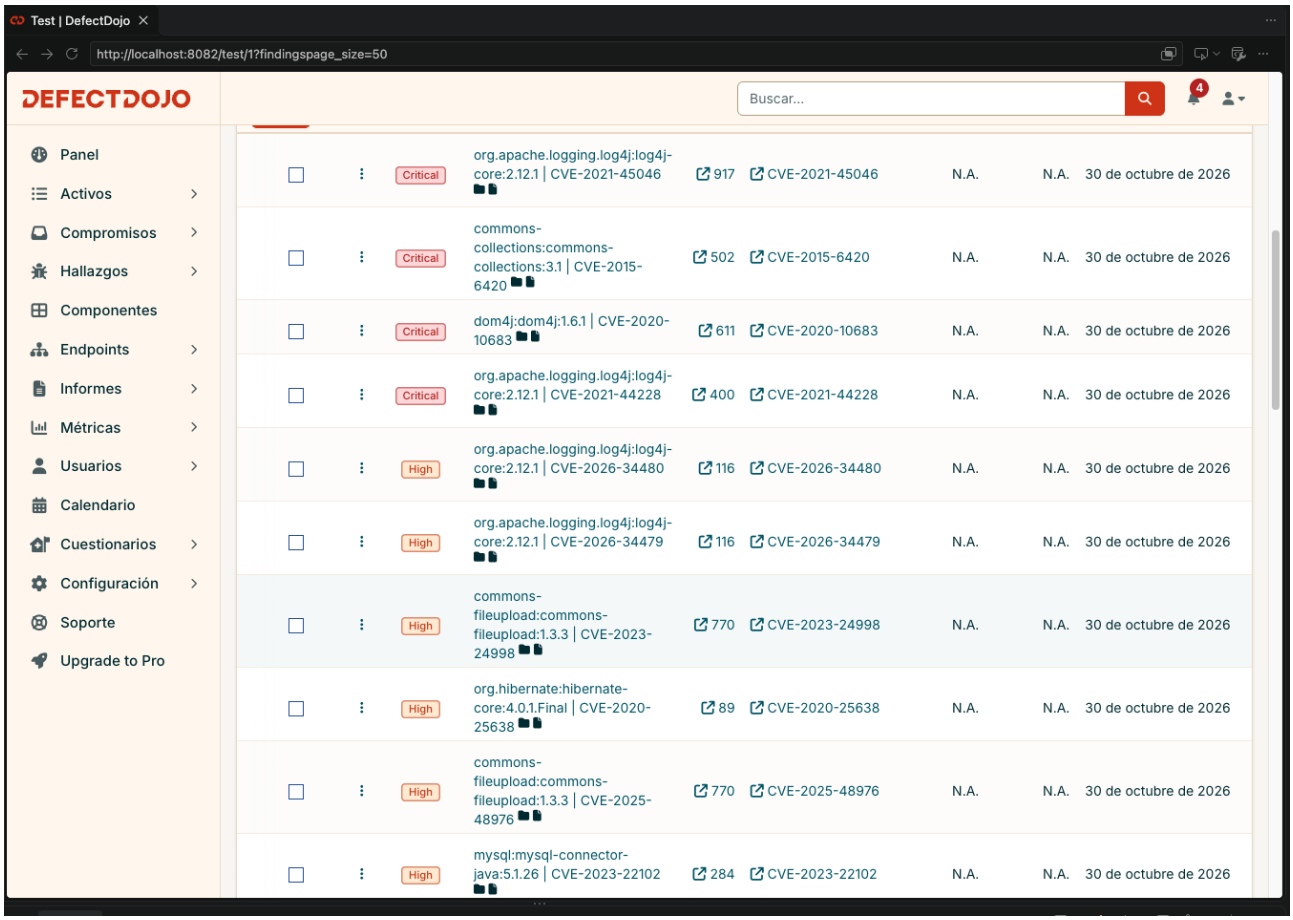



*Figura 6. Hallazgos importados en DefectDojo.*

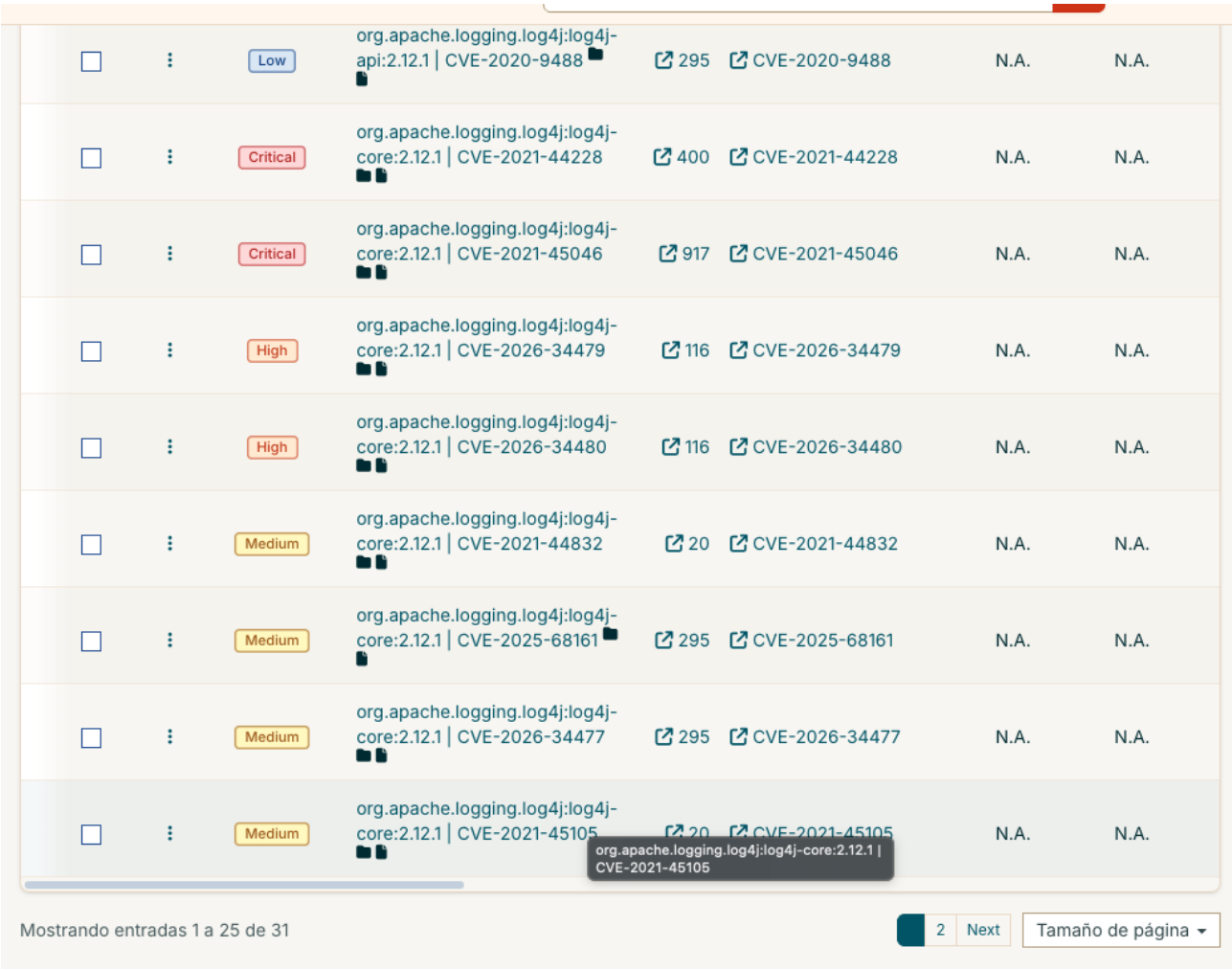



*Figura 7. Detalle de hallazgos en DefectDojo.*

## Comparación

Las dependencias vulnerables más importantes coinciden en las dos herramientas: log4j-core (Log4Shell), commons-collections, dom4j, hibernate-core, commons-fileupload, commons-io y mysql-connector-java. Las diferencias son estas:

| | **Dependency-Check (DefectDojo)** | **Dependency-Track** |
|---|---|---|
| Total | 31 hallazgos | 29 hallazgos |
| Identificadores | CVE | GHSA (alias de los CVE) |
| jquery 1.6.4 | Sí, 5 CVE y 1 issue | No aparece |
| org.json 20090211 | No aparece | Sí, 2 altas |
| log4j-api 2.12.1 | Sí, 3 CVE | No aparece |

## ¿Por qué pasa?

- **Alcance distinto.** Dependency-Check revisa también los archivos del proyecto y con el analizador RetireJS encontró jquery.min.js dentro de src/main/webapp. El SBOM de CycloneDX solo tiene las dependencias de Maven, entonces Dependency-Track no sabe que existe ese jquery.

- **Forma de buscar.** Dependency-Check adivina el CPE a partir del nombre del jar. A log4j-api le pone el CPE genérico apache:log4j y por eso le asigna CVEs que en realidad son de log4j-core (posibles falsos positivos). Para org.json no encontró CPE y no reportó nada (falso negativo). Dependency-Track con OSV busca por el purl exacto del paquete.

- **Bases de datos distintas.** NVD y OSV/GitHub Advisories no se actualizan al mismo tiempo y a veces no califican igual la severidad.

Por eso conviene usar las dos herramientas: cada una encuentra cosas que la otra no ve.

## Parte E - Pipeline con Dependency-Check

Archivo: .github/workflows/sca-dependency-check.yml. Cada línea del YAML tiene un comentario que explica lo que hace.

| # | Requisito | Cómo lo resolví |
|---|---|---|
| 1 | push, pull_request a main y manual | on: push, pull_request (branches: main) y workflow_dispatch |
| 2 | Permisos mínimos | permissions: contents: read y security-events: write |
| 3 | Java 17 | actions/setup-java con distribución temurin y java-version 17 |
| 4 | Clave del NVD como secreto | Secreto NVD_API_KEY en el repo, se usa con ${{ secrets.NVD_API_KEY }} y se pasa como variable de entorno |
| 5 | Caché de la base del NVD | actions/cache/restore al inicio y actions/cache/save al final con if: always() sobre ~/.m2/repository/org/owasp/dependency-check-data |
| 6 | HTML, XML y SARIF | -Dformats=HTML,XML,SARIF |
| 7 | SARIF a Code scanning | github/codeql-action/upload-sarif con category: dependency-check |
| 8 | Reportes como artefactos | actions/upload-artifact con target/dependency-check-report.* |
| 9 | Comentarios | Todas las líneas del YAML están comentadas |
| Extra | Job de SBOM | Segundo job que corre makeAggregateBom y guarda bom.json como artefacto |

### Decisiones del pipeline

- **Fallar por severidad:** usé -DfailBuildOnCVSS=9, entonces el job falla si hay alguna vulnerabilidad con CVSS 9 o más. Por eso las ejecuciones salen en rojo. Es lo esperado: el pipeline funciona como una barrera mientras queden vulnerabilidades críticas.

- **Publicar aunque falle:** los pasos de subir el SARIF, guardar los artefactos y guardar la caché tienen if: always(), igual que en el pipeline de Semgrep, para que se ejecuten aunque el análisis falle.

- **Caché:** en la primera versión usé actions/cache normal, pero esa acción solo guarda la caché si el job termina bien, y como el job falla a propósito nunca la guardaba. La separé en restore y save con if: always(). Con eso el análisis bajó de **27 minutos a unos 40 segundos**.

- **OSS Index desactivado:** ahora pide credenciales, entonces lo desactivé igual que en local.

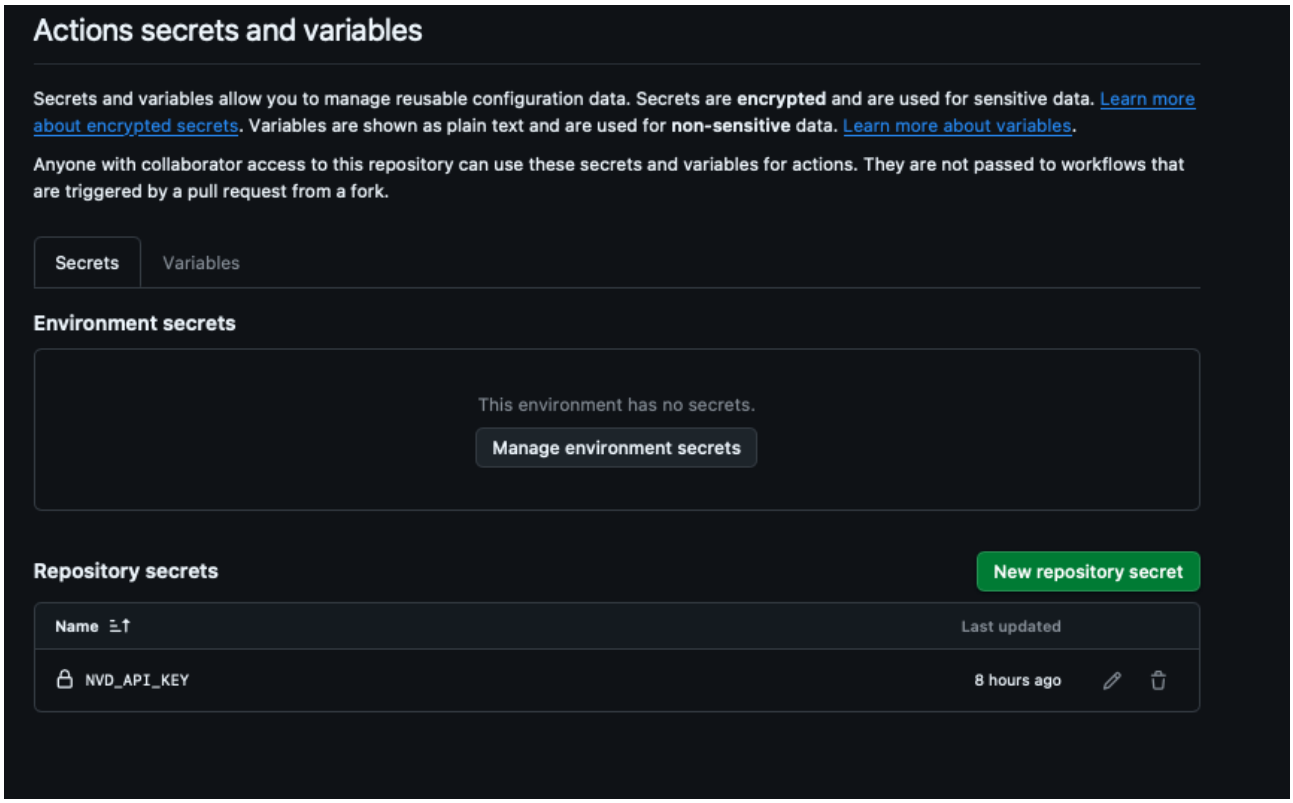



*Figura 8. Secreto NVD_API_KEY creado en el repositorio (no se muestra el valor).*

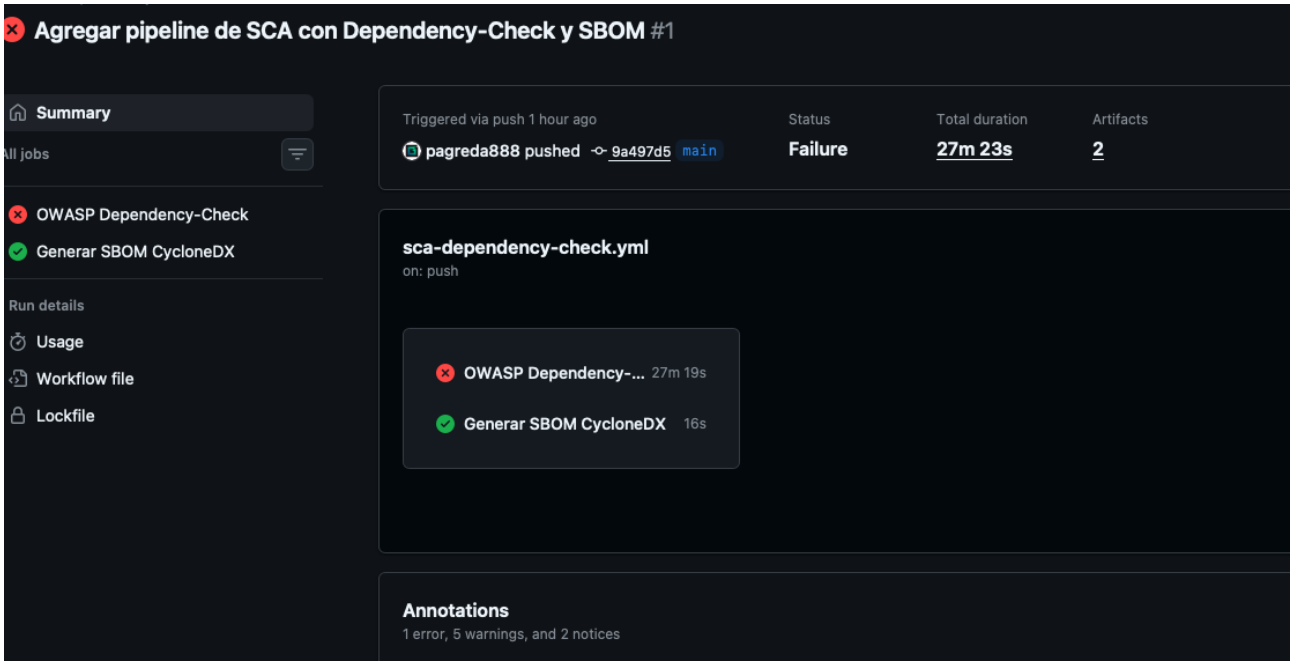



*Figura 9. Primera ejecución del workflow: los dos jobs y 2 artefactos.*

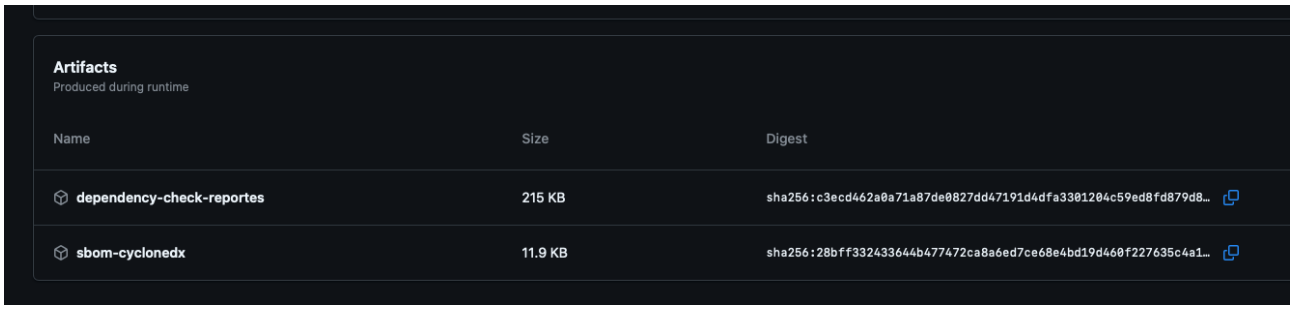



*Figura 10. Artefactos: reportes de Dependency-Check y SBOM.*

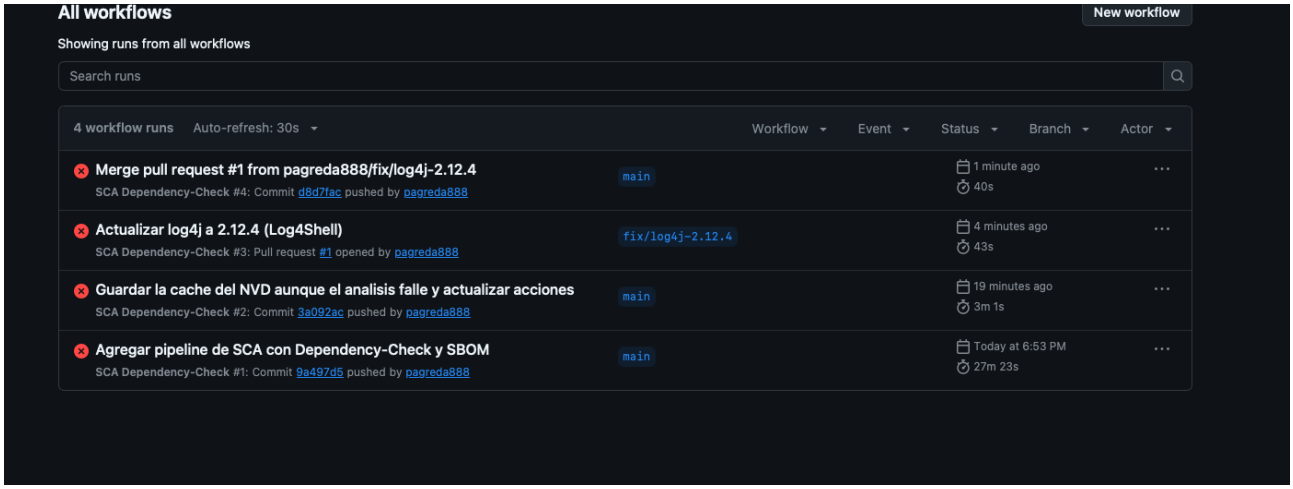



*Figura 11. Historial de ejecuciones. Con la caché pasó de 27 min a 40 s.*

## Remediación

Actualicé log4j-api y log4j-core de 2.12.1 a **2.12.4** en una rama y abrí un Pull Request hacia main: https://github.com/pagreda888/javavulnerablelab-sca/pull/1. Escogí 2.12.4 y no la última versión porque el proyecto compila para Java 7 (maven.compiler.source 1.7) y 2.12.4 es la versión corregida de la rama que todavía soporta Java 7.

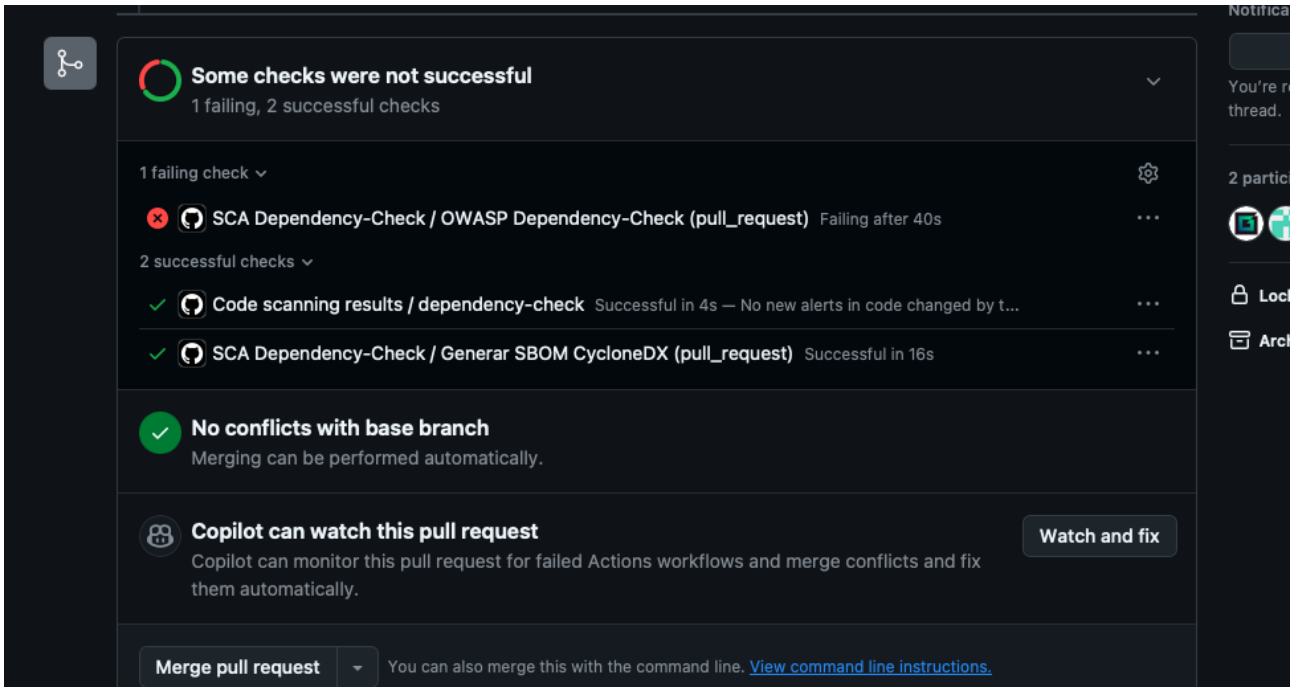



*Figura 12. Checks del Pull Request. Code scanning no reporta alertas nuevas.*

Antes del cambio, Code scanning tenía 31 alertas abiertas. Después del merge quedaron 25 abiertas y 12 se cerraron como **fixed**, entre ellas CVE-2021-44228 (Log4Shell) y CVE-2021-45046, que eran críticas. Algunos CVE recientes (CVE-2025-68161 y los CVE-2026) también afectan a 2.12.4 y volvieron a aparecer para la versión nueva, porque su corrección solo existe en versiones que piden Java 8. El job sigue fallando porque aún quedan críticas en commons-collections y dom4j, que no eran parte de esta remediación.

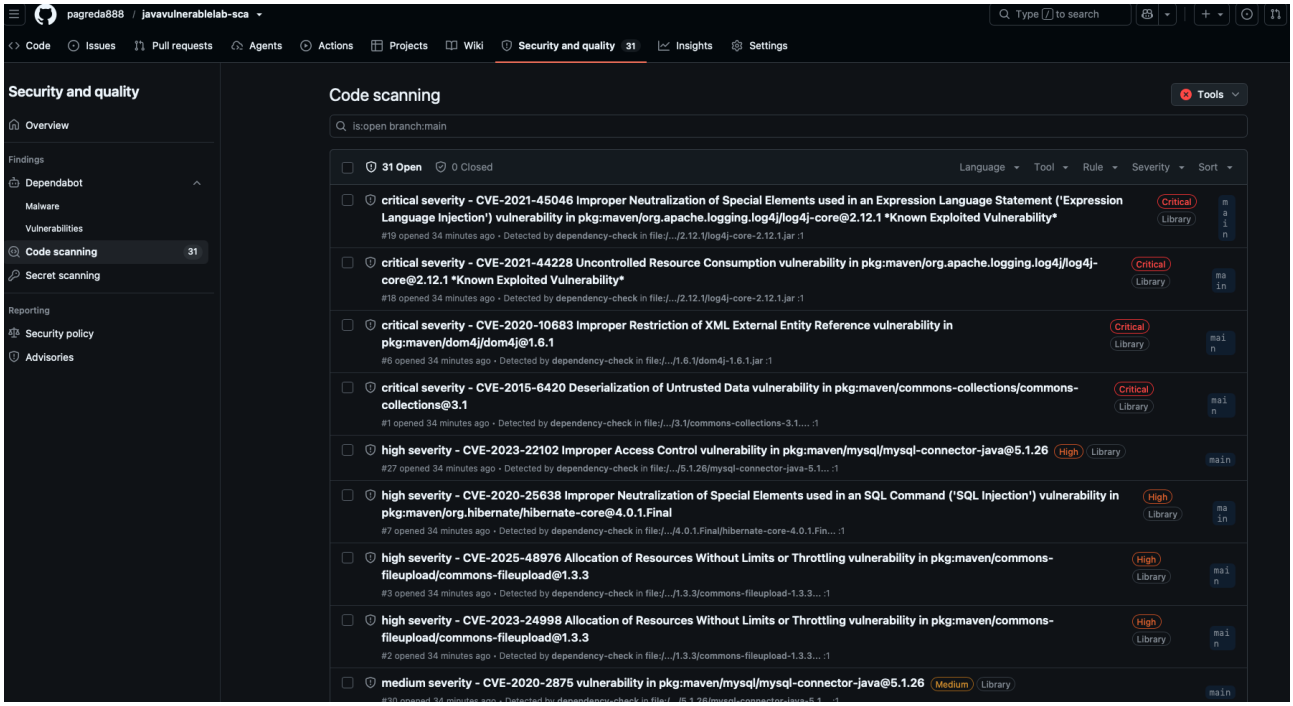



*Figura 13. Code scanning antes de la remediación (31 alertas abiertas).*

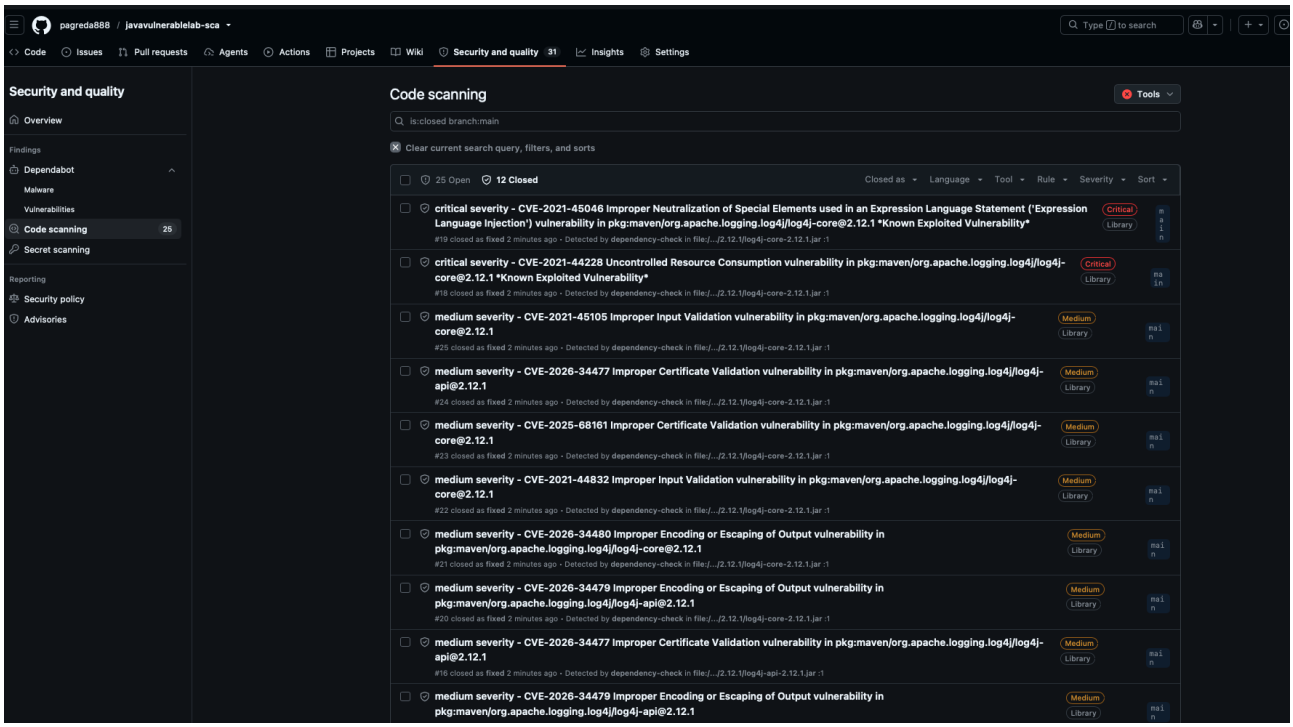



*Figura 14. Code scanning después del merge: 12 alertas cerradas como fixed.*

Alertas cerradas: [ver en GitHub]

## Parte F - Preguntas de análisis

### 19. ¿Qué diferencia hay entre SAST y SCA? ¿Por qué una aplicación necesita ambos?

SAST analiza el código fuente que escribe el equipo (por ejemplo con Semgrep) y busca errores como inyección SQL, XSS o secretos en el código. SCA analiza las librerías de terceros que usa la aplicación y las compara con bases de vulnerabilidades conocidas (NVD, OSV). Se necesitan los dos porque cubren partes distintas: el código propio puede estar bien escrito y la aplicación igual ser vulnerable por una librería, como pasa aquí con log4j. Y al revés, tener todas las librerías al día no sirve si el código propio tiene una inyección SQL. En aplicaciones modernas la mayor parte del código que se ejecuta viene de dependencias.

### 20. ¿Qué es una dependencia transitiva y por qué es difícil de controlar? Ejemplo del proyecto.

Es una dependencia que no declaro yo, sino que viene porque una de mis dependencias la necesita. Es difícil de controlar porque no aparece en el pom.xml, muchas veces no se sabe que existe, y su versión la decide la librería que la trae. Para cambiarla hay que actualizar la librería padre o forzar la versión con dependencyManagement o exclusiones. En este proyecto, dom4j 1.6.1 (CVE-2020-10683, crítica) llega por hibernate-core 4.0.1.Final, aunque nunca se declaró en el pom.xml.

### 21. ¿Qué ventaja tiene Dependency-Track frente a correr Dependency-Check solo en el pipeline?

Dependency-Check en el pipeline solo revisa el proyecto cuando hay un push o un PR. Si el código no cambia pero mañana publican un CVE nuevo para una librería que ya uso, no me entero hasta el

siguiente cambio. Dependency-Track guarda el SBOM y lo vuelve a analizar cada vez que se actualizan sus fuentes, entonces avisa de vulnerabilidades nuevas en aplicaciones que ya están en producción. Además permite ver todo el portafolio de proyectos, auditar y marcar hallazgos, configurar políticas y notificaciones.

### 22. ¿Por qué Dependency-Check y Dependency-Track pueden dar resultados distintos?

Lo vi en la Parte D. Usan fuentes distintas (NVD contra OSV/GitHub Advisories), buscan de forma distinta (CPE adivinado a partir del nombre del archivo contra purl exacto) y analizan cosas distintas: Dependency-Check revisa archivos del proyecto como el jquery.min.js, mientras que Dependency-Track solo ve lo que está en el SBOM. También influye cuándo se actualizó cada base y cómo califica la severidad cada fuente.

### 23. Si una librería vulnerable no tiene versión corregida, ¿qué alternativas hay?

- Revisar si la parte vulnerable realmente se usa. Si no es explotable en el proyecto, documentarlo y suprimir el hallazgo con justificación (archivo de supresión de Dependency-Check o el análisis de Dependency-Track).

- Aplicar mitigaciones. Por ejemplo, para Log4Shell se podía desactivar el lookup o quitar la clase JndiLookup del jar. Para commons-collections, usar una lista blanca de clases al deserializar.

- Controles alrededor de la aplicación: WAF, validar las entradas, segmentar la red y correr con privilegios mínimos.

- Cambiar a una librería alternativa que esté mantenida o, si es poco código, reemplazarla.

- Si el problema es la versión de Java, como aquí con log4j, planificar la migración para poder usar la versión corregida.

- Aceptar el riesgo formalmente, con responsable y fecha de revisión, y dejarlo monitoreado en Dependency-Track.

## Entregables

- Repositorio con el workflow: https://github.com/pagreda888/javavulnerablelab-sca

- Pull Request de la remediación: https://github.com/pagreda888/javavulnerablelab-sca/pull/1

- Reporte HTML de Dependency-Check (local): entregables/dependency-check-report.html

- SBOM: entregables/bom.json

- Capturas: carpeta entregables/capturas